

Homework 2 Programming: Bandits

CS/Stat 184(0): Introduction to Reinforcement Learning

(Due October 13, 2024 at 11:59 pm ET)

Note on Submission

All files related to the programming assignment should be submitted on Gradescope to “HW2 Programming” only.

3 Programming

3.1 Instructions

This programming assignment uses Python 3. It is recommended to use Anaconda to install Python as well as the required libraries. Something like the follow should work:

- `conda create -n stat184hw2 -c conda-forge matplotlib==3.9.2 numpy==2.1.1 python=3.10.13`
- `conda activate stat184hw2`

Note that the above exact versions of Python and the packages have been verified to work here. If not using this installation method and you have issues, try using these versions. Feel free to ask for help through this process!

3.2 Introduction

For this assignment, you will implement 6 different algorithms for the Multi-Armed Bandit (MAB) problem, and compare their performances.

The setup is as follows: The MAB has K arms, and horizon T . At time t ($t = 0, 1, \dots, T-1$), the reward obtained by pulling arm k ($k = 1, 2, \dots, K$) is a Bernoulli random variable with mean μ_k , where $\mu_1, \mu_2, \dots, \mu_K \in [0, 1]$ are fixed parameters. We use a_t to denote the arm pulled at time t , and r_t to denote the observed reward at time t .

The default values are $K = 5, T = 100, \boldsymbol{\mu} = (0.3, 0.5, 0.6, 0.65, 0.7)$. Each algorithm is run $R = 30$ times. Feel free to play around with those, but only submit the result with the values specified above.

3.3 Deliverables

This project consists of the following three files:

- `main.py`: the main file where each algorithm runs R times, each time for T steps. Then the average cumulative regrets, the 5th and the 95th percentiles at each time step are plotted. You should not change this file.
- `env_MAB.py`: the file that contains the MAB class. You should not change this file.
- `algorithms.py`: the file that contains all the 6 algorithms as classes. All student code goes here.

You can find the three files on Canvas. Please implement the 6 algorithm classes in `algorithms.py`. Then run `python main.py` command in your terminal. This should generate and save a plot with 6 subplots, each showing the performance of a certain algorithm. Create a new file called `Interpretation.md`. In this file, explain what you see in the performance comparison plot. See details on plotting and interpretation in Section 3.5.

Zip your code (with the 3 python files, the generated png file, and the `Interpretation.md` file), and submit it on Gradescope.

3.4 Algorithms implementation

3.4.1 General tips

- For each class, you need to implement the `play_one_step` method (necessary), as well as the `reset` and `__init__` methods (optional, useful for some algorithms with parameter updates). Feel free to add other helper methods.
- You might find these methods from MAB class in `env_MAB.py` useful:
 - `MAB.get_K`: returns K , the number of arms in an MAB object.
 - `MAB.get_T`: returns T , the horizon.
 - `MAB.get_record`: returns a $K \times 2$ array, containing the number of previous 0 (failures) and 1 (successes) for each arm in this MAB.
- For other methods in MAB class:
 - `MAB.pull`: pull the given arm, and record the reward as well as regret (unseen by the algorithm). You should call this method exactly once in your `play_one_step` method.
 - `MAB.reset`: reset the MAB and clear the history. This method is called in the `reset` method for each algorithm. Do not call it anywhere else.
 - `MAB.get_regrets`: get the regret history. This is only for performance evaluation. You should not call this method anywhere.
- Each time you need to find an argmax or argmin and there are ties, randomly choose one of them. You may find the `random_argmax` function in `algorithms.py` useful.

3.4.2 Pure Exploration [4 points]

Implement this algorithm in the Explore class. Pull each arm an equal number of times. This can be accomplished in a number of ways. For this assignment, please implement the algorithm as follows: at each step, pull the arm that has been pulled the least number of times, and ties are broken uniformly at random. This also addresses the case when K does not divide T .

3.4.3 Pure Greedy [4 points]

Implement this algorithm in the Greedy class. Pull each arm once (in any order). At each time step after that, pull the greedy arm (i.e., the arm with the highest observed reward at the current time step). Ties are broken uniformly at random.

3.4.4 Explore-then-Commit [5 points]

Implement this algorithm in the ETC class. During the exploration phase, pull each arm N_e times, where

$$N_e = \left\lceil \left(\frac{T \sqrt{\ln(2K/\delta)/2}}{K} \right)^{2/3} \right\rceil$$

and $\delta = 0.05$ (by default). For this assignment, please implement the exploration phase as follows: Among all arms which have been pulled less than N_e times, randomly pick one and pull it. Continue until all arms have been pulled N_e times.

At each time step after that, pull the best empirical arm (i.e. the arm with the highest empirical mean during the exploration phase). Ties are broken uniformly at random.

3.4.5 ϵ -greedy [5 points]

Implement this algorithm in the Epgreedy class. First pull each arm once. Then for each later $t \geq K$:

- With probability ϵ_t , randomly choose an arm to pull (explore).
- With probability $1 - \epsilon_t$, pull the greedy arm (exploit),

where $\epsilon_t = \left(\frac{K \ln(t)}{t} \right)^{1/3}$, and ties are broken at random.

3.4.6 Upper Confidence Bound (UCB) [7 points]

During each iteration of the UCB algorithm, we calculate an upper confidence bound for each arm, which is the empirical reward estimate plus a term that depends on the number

of times the arm has been played. We then play the arm with the highest upper confidence bound. More formally, the empirical reward estimate for k -th arm at time t is

$$\hat{\mu}_t(k) := \frac{\sum_{s=0}^{t-1} 1_{\{a_s=k\}} r_s}{\sum_{s=0}^{t-1} 1_{\{a_s=k\}}}, \quad k = 1, 2, \dots, K, \quad t = 0, 1, \dots, T-1, \quad (1)$$

and the arm played at time t is

$$a_t = \arg \max_{k \in 1, \dots, K} \left(\hat{\mu}_t(k) + \sqrt{\frac{\ln(KT/\delta)}{\sum_{s=0}^{t-1} 1_{\{a_s=k\}}}} \right), \quad t = 0, 1, \dots, T-1,$$

where $\hat{\mu}_t(k)$ is defined in equation (1), and ties are broken uniformly at random. Moreover, we set the confidence δ to be 0.05.

Note the the denominators in the equations above are equal to 0 if the k -th arm has never been played. When implementing the UCB algorithm, we set the fractions in the equations above to be (positive) infinite when their denominator are equal to 0. As a result, we will prioritize arms that have never been explored yet, and effectively the algorithm starts by playing each of the K arms once.

Algorithm 1 shows the pseudo-code for the UCB algorithm.

Algorithm 1 UCB

Input: Number of arms K , horizon $T \geq K$, confidence $\delta > 0$

for $t = 0, 1, \dots, T-1$ do

 Play arm

$$\arg \max_{k \in 1, \dots, K} \left(\hat{\mu}_t(k) + \sqrt{\frac{\ln(KT/\delta)}{\sum_{s=0}^{t-1} 1_{\{a_s=k\}}}} \right),$$

 where ties are broken uniformly at random

end for

3.4.7 Thompson Sampling [10 points]

Thompson Sampling is simple to implement, yet has strong empirical performance. We start by imposing independent uniform priors on $\mu_1, \mu_2, \dots, \mu_K$. We can express these uniform priors as Beta(α, β) priors with $\alpha = \beta = 1$. As a reminder, the Beta(α, β) distribution on μ_k is given by:

$$\pi(\mu_k | \alpha, \beta) = \frac{\mu_k^{\alpha-1} (1 - \mu_k)^{\beta-1}}{B(\alpha, \beta)}$$

Where B is the beta function:

$$B(\alpha, \beta) = \int_0^1 x^{\alpha-1} (1-x)^{\beta-1} dx = \frac{(\alpha-1)!(\beta-1)!}{(\alpha+\beta-1)!}$$

For the Thompson sampling algorithm, during each iteration, we sample a success probability estimate $\hat{\mu}_k$ from the posterior distribution of μ_k for each $k = 1, 2, \dots, K$. We play the arm with the largest success probability estimate, and ties are broken uniformly at random. After observing the reward, we update the posterior distribution for the arm that we have just played. As a reminder, for positive integers α, β , the Beta distribution is the conjugate prior of the binomial distribution, which just means it is very easy to update. In particular, if we start with a $\text{Beta}(\alpha, \beta)$ prior and observe j successes out of n samples, the posterior distribution of μ_k will be given by:

$$\begin{aligned} P(\mu_k | j, \alpha, \beta) &\propto P(j | \mu_k) \pi(\mu_k | \alpha, \beta) \\ &\propto \mu_k^j (1 - \mu_k)^{n-j} \mu_k^{\alpha-1} (1 - \mu_k)^{\beta-1}, \end{aligned}$$

which is a $\text{Beta}(\alpha + j, \beta + n - j)$ distribution.

Algorithm 2 shows the pseudo-code for Thompson sampling.

Algorithm 2 Thompson Sampling

Input: Number of arms K , horizon $T \geq K$
Initialize $\alpha = \beta = \mathbf{1} \in \mathbb{R}^K$,
for $t = 0, 1, \dots, T - 1$ do
 for $k = 1, 2, \dots, K$ do
 Sample $\hat{\theta}_k \sim \text{Beta}(\alpha_k, \beta_k)$
 end for

 Play arm $a_t = \arg \max_{k \in 1, \dots, K} \hat{\theta}_k$, where ties are broken uniformly at random

 Update the belief for arm a_t : $(\alpha_{a_t}, \beta_{a_t}) \leftarrow (\alpha_{a_t} + r_t, \beta_{a_t} + 1 - r_t)$
end for

3.5 Visualization and Interpretation [10 points]

After implementing the algorithms and executing `main.py`, you will find a plot named `performances.png` in the same folder as `main.py`. This plot visualizes the performance for each of the algorithms you have just implemented. The x-axis for each subplot represents time step (i.e. t), and the line plot shows the average cumulative regrets over $R = 30$ runs. As a reminder, for each run, the cumulative regret at time t is defined as

$$R(t) := \sum_{s=0}^t \left(\max_{k \in 1, 2, \dots, K} \mu_k - \mu_{a_s} \right), \quad t = 0, 1, 2, \dots, T - 1,$$

where a_s is the arm pulled at time step s . Furthermore, the shadowed area in each subplot represents the range between the 5th and the 95th percentiles of the cumulative regrets at each time step.

Your task is to compare the performances of all the algorithms. Create a new file called Interpretation.md. In this file, explain what you see in the performance comparison plot. How do the algorithms compare? Which is good/bad and why? Which is the best? Is an algorithm always good/bad? Feel free to be flexible (i.e. you don't have to answer the previous questions in order).

Good Luck!